MINISTRY OF EDUCATION AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatics

REPORT

Laboratory Work No. 2

Topic: **Cryptanalysis of Monoalphabetic Ciphers**

Breaking a monoalphabetic substitution cipher with frequency analysis
Variant 7

Author: Bîtca Cătălin
Group: FAF-242

Chișinău, 2026

1. Short theory

1.1. Monoalphabetic ciphers

A monoalphabetic substitution cipher replaces every letter of the plaintext with another letter, always using the same rule for the whole message. If **e** is encrypted as **V** once, it is encrypted as V everywhere in the text. The Caesar cipher from Lab 1 is the simplest example, but in general the substitution can be any permutation of the alphabet, so for English there are $26! \approx 4 \cdot 10^{26}$ possible keys. Trying all of them is impossible, but the cipher can still be broken easily, because it does not hide the statistics of the language.

1.2. Frequency analysis

In every language some letters are used much more often than others. In English the most frequent letters are E (12.70%), T (9.06%), A (8.17%), O (7.51%), I (6.97%) and N (6.75%), while J, X, Q and Z appear in less than 0.2% of cases (Table 2.2 of the lab). A monoalphabetic cipher only renames the letters, so the ciphertext has exactly the same frequencies, only under other names. If the ciphertext is long enough, the most frequent cipher letter is very probably the image of E, the next ones of T, A, O and so on.

Frequencies alone are not enough to decide every letter, because letters with close frequencies (for example S, H, R or C, U, M) can easily change places in a particular text. That is why the attack also uses other properties of the language:

- the most common short words: “the”, “and”, “of”, “to”, “in”, and the only one-letter words “a” and “I”;
- frequent pairs and triples of letters (TH, HE, IN, ER, AN; THE, AND, ING, ION);
- doubled letters, which are mostly SS, LL, EE, TT, OO, FF;
- the apostrophe: a single letter after it is usually ’s or ’t;
- the context: once part of the letters is known, many words can be read almost completely and the missing letters can be guessed.

The attack is done step by step: each guess is substituted in the whole text, and if new readable words appear the guess is kept, otherwise it is changed. This needs human decisions, so the computer is used only to count the frequencies and to apply the substitutions quickly.

2. The task

An encrypted message was intercepted, and it is known that it was encrypted with a monoalphabetic cipher. Assuming that the message is written in English, I had to find the original text using frequency analysis. Only the letters were encrypted; spaces, digits and punctuation were left as they are. I solved **variant 7**, which has 2842 letters:

```
"OTWN tgo X rviv pwinssxgj xg wqv Pduivzv Ungwxcc'p jtiovgp tw wqv Ktwxhtgtgo rv rvgw cinz
wnuxh wn wnuxh ztikvsxgj tw wqv xgjvgdxwf wqtw zvgpqnrvo xg ktixndp vgwviuixpvp, wxss Otwn
jtkv vyuivppxng wn qxp rtiztozxitwxng cni wqnpv zvg rqn htg vyusnxw rqtw tiv htssvo 'hxuqvip.'
"Pn rinwv Svng Atwwxpwt Tsaviwx gvti wqv avjxggxgj nc wqv pdhhxghw adwpdjjvpwxkv rnil
wqtw vtigvo qxz wqv wxwsv nc Ctwqvi nc RvpwvigHifuwnsnjf. Tsaviwx rtp wqv cxipw nc t jindu
nc rixwvip rqn, vsvzvgw afvsvzvgw, ovkvsnuvo t wfuv nc hxuqvi wn rqxhq znpw nc wnotf'p pfpwvzp
nchifuwnjituqf avsngj. Wqv puvhxvp xp unsftsuqtavwxh pdapwxwdwxng.Xw rtp wqv tztwvdip nc
hifuwnsnjf rqn hivtwvo wqv puvhxvp. Wqvuincvppxngtsp, rqn tsznpw hviwtxgsf pdiutppvo wqvz xg
hifuwtgtsfwxhvyuviwxpv, hnghvgwitwvo ng wqv onrg-wn-vtiwq uinasvzp nc wqv pfpwvzpwqtw
rviv wqvg xg dpv adw tiv gnr ndwotwvo. Wqv tztwvdip, dgcvwwvivo wnwqvpv ivtsxwxvp, pntivo
xgwn wqv vzufivtg nc wqvnif. Wqviv rviv cndirqnpv wqndjqw wnnl rxgjp: t ctzndp tihqxwvhw, tg
xgwvssvhwdts hsvixh, tgvhhsvpxtpwxhts hndiwxvi, tgo t gtwdits phxvgwxpw.Wqv tihqxwvhw rtp
Tsaviwx, t ztg rqn, uviqtup avwwvi wqtg tgfngvvyhvuw Svngtion ot Kxghx, vuxwnzxmvp wqv
Ivgtxpptghv xovts nc wqvdgxkvipts ztg. Anig xg 1404, wqv xssvjxwxztwv adw ctknivo png nc t
ctzxsfnc ixhq Csnivgwxgv zvihqtgwp, Tsaviwx vgenfvo vywitnioxgtif xgwvssvhwdtstgo twqsvwxh
tuwxwdovp. Qv utxgwvo, hnzunpvo zdpxh, tgo rtp ivjtiovotp ngv nc wqv avpw nijtgxpwp nc qxp otf.
Rixwxgjp undivo cinz qxp uvg. QxpOv Iv Tvoxcxhtwnixt, wqv cxipw uixgwvo annl ng
tihqxwvhwdiv, rixwwvg rqxsvJnwqxh hqdihqvp rviv pwxss avxgj adxsw, qvsuvo pqtuv wqv
wqndjqwp ncwqnpv rqn adxsw pdhq dwwvisf gng-Jnwqxh pwidhwdivp tp Pw. Uvwvi'p Atpxsxhtxg
Inzv. Ethna Adihlqtiow, tdwqni nc wqv hstppxh Wqv Hxkxsxmtwxng nc wqvIvgtxppttghv xg Xwtsf,
pxgjsvo ndw Tsaviwx tp ngv nc wqv widsf tss-pxovozvg rqn wnrvi tankv wqvxi gdzvindp ztgf-pxovo
hngwvzunitixvp. Tgotgnwqvi jivtw Ivgtxpptghv phqnsti, Enqg Pfzngop, ovhstivo wqtw
"Qvuivpvgwp wqv puxixw nc wqv 15wq hvgwdif tw xwp kvif avpw."Tzngj qxp cixvgop rtp wqv
ungwxcxhts pvhivwtif, Svngtion Otwn, ngv ncwqv svtigvo zvg nc qxp tjv, rqn odixgj wqtw zvznitasv
pwinss xg wqvKtwxhtg jtiovgp aindjqw wqv hngkviptwxng tindgo wn hifuwnsnjf. "Fnd'kvtsrtfp avvg
xgwvivpwvo xg wqvpv pvhivwp nc gtwdiv," Otwn ptxo. "Rqtw onfnd wqxgl nc wqvpv ovhxuqvivip?
Qtkv fnd wixvo fndi qtgo tw xw, tp zdhqtp fnd lgnr qnr wn?"Tsaviwx pzxsvo. Qv lgvr wqtw Otwv'p
odwxvp xghsdovo hxuqvip (xw rtpavcniv wqv hdixt qto t pvutitwv hxuqvi pvhivwtif). "Fnd'iv wqv
qvto nc wqvutuvs pvhivwtixtw," qv wvtpvo. "Hndso xw av wqtw fnd qto wn dpv wqvpvwqxgjp t cvr
wxzvp xg ztwwvip nc jivtw xzuniwtghv wn Qxp Qnsxgvpp?""Wqtw'p rqf X aindjqw xw du," Otwn
ivusxvo htgoxosf. "Tgo avhtdpv ncwqv unpw X qtkv, X rtgw wn av tasv wn on xw zfpvsc rxwqndw
qtkxgj wn dpvndwpxov xgwviuivwvip. Cni rqvg wqvf aixgj zv svwwvip xg hxuqvi xgwvihvuwvoaf
puxvp, xw'p gn enlxgj ztwwvi. Pn usvtpv-xc fnd'kv wqndjqw du tgf gvrxovtp qtkxgj wn on rxwq
wqxp adpxgvpp, wvss zv tandw wqvz." Pn Tsaviwxuinzxpvo wqtw qv rndso on pnzv rnil ng xw pn
wqtw Otwn rndso pvv wqtwxw rtp uincxwtasv wn qtkv tplvo qxz, tgo wqv ivpdsw rtp wqv vpptf
wqtw qvrinwv xg 1466 ni vtisf 1467, rqvg qv rtp 62 ni 63.Qv xzusxvo wqtw qv wqndjqw du wqv
xovt nc civbdvghf tgtsfpxp tss afqxzpvsc, adw wqv hnghvuwxng wqtw qv pvw cniwq xp cti wnn
ztwdivo cni wqtw.Gvkviwqvsvpp, qxp ivztiltasf sdhxo Stwxg vpptf, wnwtsxgj tandw 25ztgdphixuw
utjvp, hngpwxwdwvp wqv Rvpw'p nsovpw vywtgw wvyw nghifuwtgtsfpxp.
```

Ciphertext of variant 7

3. Implementation

The lab suggests using an online frequency analysis page. Instead, I wrote a small tool in **Python 3**, `frequency_analysis.py`, which does the same job in the terminal: it counts the letter frequencies of the ciphertext, compares them with English, shows the most common short words, and lets me try substitutions one by one and immediately see the text. The decisions about which letter is which are still made by me, the tool only does the counting and the replacing. When the program starts it reads `ciphertext.txt`, prints the frequency table and waits for commands (Figure 1).

3.1. Counting the letter frequencies

```
def letter_frequencies(text):
    counts = Counter(ch.upper() for ch in text if ch in LETTERS)
    total = sum(counts.values())
    table = [(letter, counts[letter], 100 * counts[letter] / total) for letter in UPPER]
    return sorted(table, key=lambda row: (-row[1], row[0]))


def english_order():
    return sorted(UPPER, key=lambda letter: -ENGLISH[letter])
```

`letter_frequencies` counts every letter of the text without caring about upper or lower case (`Counter` from the standard library), computes the percentage of each letter from the total number of letters and sorts the table from the most frequent letter to the least frequent. Letters that do not appear get 0. `english_order` sorts the English letters by the frequencies from Table 2.2 of the lab (stored in the `ENGLISH` dictionary), so the two orders can be printed side by side.

3.2. Finding the most common short words

```
def common_words(text, length, top=5):
    words = [word.upper() for word in re.findall(r"[A-Za-z]+", text) if len(word) == length]
    return Counter(words).most_common(top)
```

The regular expression `[A-Za-z]+` splits the text into words (sequences of letters), the words with the given length are converted to uppercase and counted. The `words` command prints the top one-, two- and three-letter words, which are the best hints after the letter frequencies.

3.3. Showing the text with the current substitutions

```
def apply_key(text, key):
    result = ""
    for ch in text:
        if ch in LETTERS and ch.upper() in key:
            result += key[ch.upper()]
        elif ch in LETTERS:
            result += ch.upper()
        else:
            result += ch
    return result
```

The key is a dictionary from a cipher letter to a plaintext letter, for example `{"V": "e"}`. I used the same convention as the example in the lab: letters that are already decrypted are shown in **lowercase**, and letters that are still unknown stay as **UPPERCASE** cipher letters. Everything that is not a letter (spaces, digits, punctuation) is copied unchanged.

3.4. Adding substitutions

```
def add_pairs(key, command):
    pairs = command.split()
    for pair in pairs:
        if not re.fullmatch(r"[A-Za-z]=[A-Za-z]?", pair):
            return f"Invalid command '{pair}'. Use CIPHER=plain, for example V=e (V= removes
it). Type help."
    new_key = dict(key)
    for pair in pairs:
        cipher, plain = pair[0].upper(), pair[2:].lower()
        new_key.pop(cipher, None)
        if not plain:
            continue
        for other, value in new_key.items():
            if value == plain:
                return f"'{plain}' is already the image of {other}. Remove it first with
{other}="
        new_key[cipher] = plain
    key.clear()
    key.update(new_key)
    return None
```

A command like `V=e W=t` adds one or more substitutions at once, and `V=` removes one. First every pair is checked with a regular expression (one cipher letter, `=`, at most one plain letter). Then the pairs are applied to a copy of the key. Because the cipher is monoalphabetic, two different cipher letters can never stand for the same plain letter, so if the plain letter is already used the command is refused with a message that says which cipher letter already has it. The real key is changed only if all the pairs are valid.

3.5. Final decryption

```
def decrypt(text, key):
    result = ""
    for ch in text:
        if ch in string.ascii_uppercase:
            result += key[ch].upper()
        elif ch in string.ascii_lowercase:
            result += key[ch.upper()]
        else:
            result += ch
    return result
```

When the key is complete, the `plain` command prints the decrypted text. Here, unlike in `apply_key`, the original case of each letter is kept, so the result looks like a normal text.

I also wrote a few unit tests (`test_frequency_analysis.py`) for the counting, the short words, the lowercase/uppercase convention, the validation of the commands and the final decryption of variant 7. All 7 tests pass.

Figures 1–3 show the tool running on my variant.

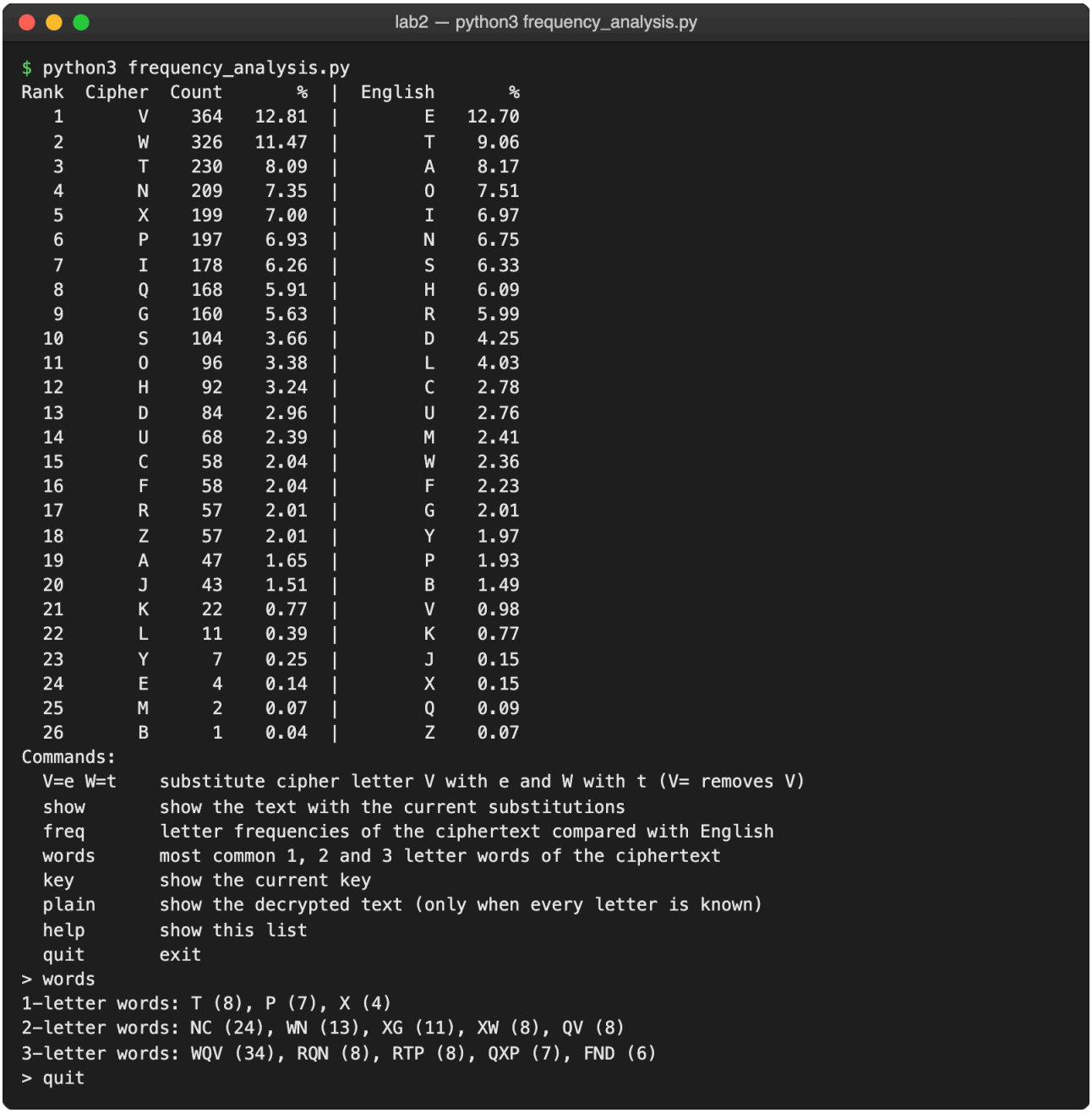


Figure 1. *Start of the program: the frequency table compared with English, the list of commands and the* `words` *command*



Figure 2. *The text after the first substitution (V = e, W = t): decrypted letters are lowercase, unknown letters are uppercase*

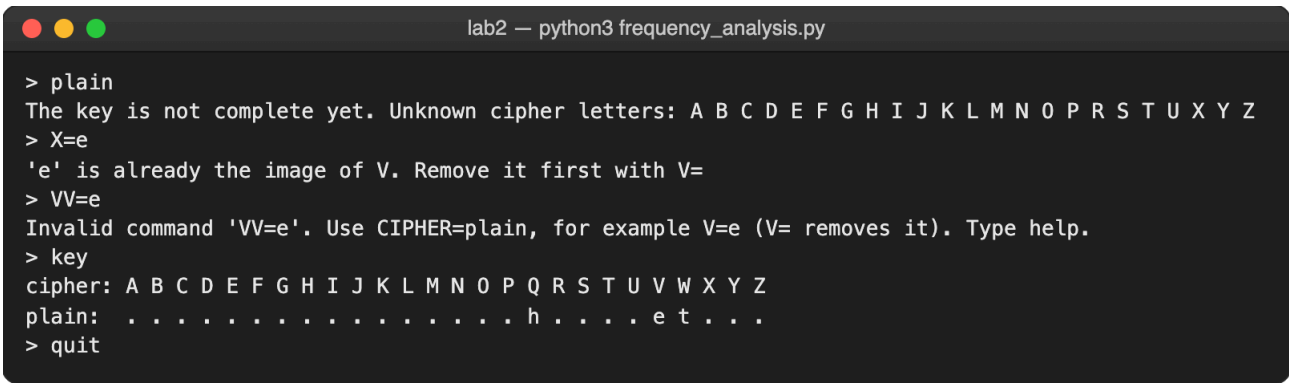


Figure 3. *Messages for an incomplete key, a plain letter that is already used and a wrong command, and the* `key` *command*

4. Breaking the cipher

4.1. Letter frequencies of the ciphertext

The first step is to count how many times every letter appears in the ciphertext (Table 1).

Letter	A	B	C	D	E	F	G	H	I	J	K	L	M
Count	47	1	58	84	4	58	160	92	178	43	22	11	2
%	1.65	0.04	2.04	2.96	0.14	2.04	5.63	3.24	6.26	1.51	0.77	0.39	0.07
Letter	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Count	209	96	197	168	57	104	230	68	364	326	199	7	57
%	7.35	3.38	6.93	5.91	2.01	3.66	8.09	2.39	12.81	11.47	7.00	0.25	2.01

Table 1. *Letter frequencies in the intercepted ciphertext (variant 7)*

For convenience, Table 2 shows the same letters sorted in decreasing order of frequency, next to the English letters sorted the same way.

Rank	1	2	3	4	5	6	7	8	9	10	11	12	13
Cipher	V	W	T	N	X	P	I	Q	G	S	O	H	D
%	12.81	11.47	8.09	7.35	7.00	6.93	6.26	5.91	5.63	3.66	3.38	3.24	2.96
English	E	T	A	O	I	N	S	H	R	D	L	C	U
%	12.70	9.06	8.17	7.51	6.97	6.75	6.33	6.09	5.99	4.25	4.03	2.78	2.76
Rank	14	15	16	17	18	19	20	21	22	23	24	25	26
Cipher	U	C	F	R	Z	A	J	K	L	Y	E	M	B
%	2.39	2.04	2.04	2.01	2.01	1.65	1.51	0.77	0.39	0.25	0.14	0.07	0.04
English	M	W	F	G	Y	P	B	V	K	J	X	Q	Z
%	2.41	2.36	2.23	2.01	1.97	1.93	1.49	0.98	0.77	0.15	0.15	0.09	0.07

Table 2. *Ciphertext letters and English letters, both sorted by frequency*

Figure 4 shows both distributions. They look almost the same: one very high bar, a group of eight letters between about 6% and 8%, and a long tail of rare letters. This already shows that the text is English and that it was encrypted with a monoalphabetic substitution (a polyalphabetic cipher would make the ciphertext frequencies much flatter).

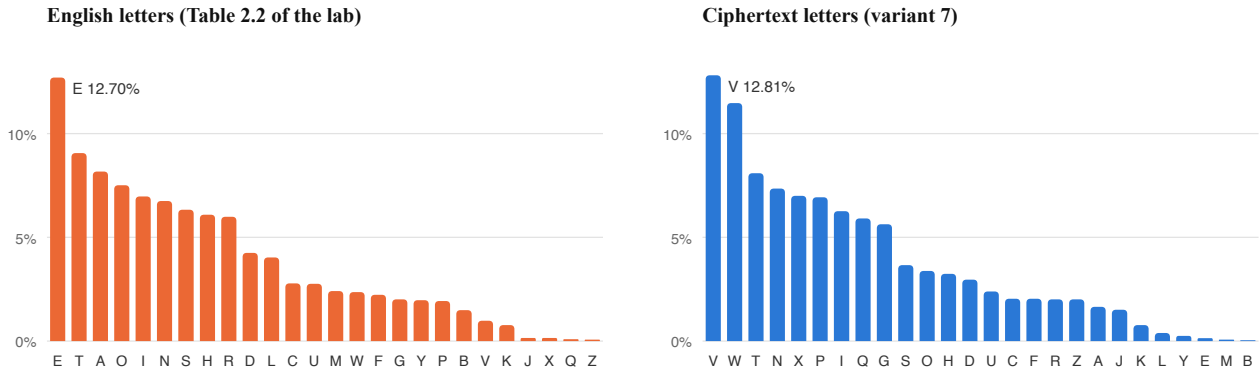


Figure 4. *Letter frequencies of English (left) and of the ciphertext (right), sorted in decreasing order (same scale, 0–14%)*

4.2. Step-by-step substitution

Below are the steps I made, in the same order as I did them. After every step I show the first lines of the text: the letters decrypted in this step are shown in **blue bold**, the letters from previous steps are plain lowercase and the unknown cipher letters are gray UPPERCASE.

Step 1: V → e, W → t

The two most frequent letters of the ciphertext are **V** (12.81%) and **W** (11.47%), clearly ahead of all the others, just like **E** (12.70%) and **T** (9.06%) in English. So I assumed that V is the image of **e** and W is the image of **t**.

Command in the tool: `V=e W=t`. First lines of the text after this step:

```
"OTtN TGO X ReIe PtINSSXGJ XG tQe PDUIeZe UNGtXCC'P JTIOeGP Tt tQe KTtXHTGTGO Re ReGt CINZ
tNUXH tN tNUXH ZTIKeSXGJ Tt tQe XGJeGDXtF tQTt ZeGPQNReO XG KTIXNDP eGteIUIXPeP, tXSS OTtN
JTKe eYUIePPXNG tN QXP RTIZTOZXITtXNG CNI tQNPe ZeG RQN HTG eYUSNXt RQTt TIe HTSSeO 'HXUQeIP.'
"PN RINte SeNG ATttXPtT TSAeItX GeTI tQe AeJXGGXGJ NC tQe PDHHXGHt ADtPDJJePtXKe RNIL
tQTt eTIGeO QXZ tQe tXtSe NC CTtQeI NC RePteIGHIFUtNSNJF. TSAeItX RTP tQe CXIPt NC T JINDU
NC RIXteIP RQN, eSeZeGt AFeSeZeGt, OeKeSNUeO T tFUe NC HXUQeI tN RQXHQ ZNPt NC tNOTF'P PFPteZP
```

Step 2: Q → h

The most frequent 3-letter word is `WQV`, which appears 34 times. After the first step it reads `tQe`, and the most common 3-letter word in English is “the”, so Q is **h**. Q is also 8th by frequency (5.91%), the same place as H in English (6.09%).

Command in the tool: `Q=h`. First lines of the text after this step:

```
"OTtN TGO X ReIe PtINSSXGJ XG the PDUIeZe UNGtXCC'P JTIOeGP Tt the KTtXHTGTGO Re ReGt CINZ
tNUXH tN tNUXH ZTIKeSXGJ Tt the XGJeGDXtF thTt ZeGPhNReO XG KTIXNDP eGteIUIXPeP, tXSS OTtN
JTKe eYUIePPXNG tN hXP RTIZTOZXITtXNG CNI thNPe ZeG RhN HTG eYUSNXt RhTt TIe HTSSeO 'HXUheIP.'
"PN RINte SeNG ATttXPtT TSAeItX GeTI the AeJXGGXGJ NC the PDHHXGHt ADtPDJJePtXKe RNIL
thTt eTIGeO hXZ the tXtSe NC CTtheI NC RePteIGHIFUtNSNJF. TSAeItX RTP the CXIPt NC T JINDU
NC RIXteIP RhN, eSeZeGt AFeSeZeGt, OeKeSNUeO T tFUe NC HXUheI tN RhXHh ZNPt NC tNOTF'P PFPteZP
```

Step 3: T → a, X → i, P → s

The ciphertext has three one-letter words: `T` (8 times), `P` (7 times) and `X` (4 times). In English only “a” and “I” are one-letter words. T is the 3rd most frequent letter (8.09%, while A has 8.17%), so T is **a**, and X (7.00%, I has 6.97%) is **i**. P is never a real word: it always comes right after an apostrophe, for example `UNGWXCC'P` or `WNOTF'P`. That is the English possessive ’s, so P is **s**. This also fits the doubled letters: `PP` appears 12 times and “ss” is very common in English.

Command in the tool: `T=a X=i P=s`. First lines of the text after this step:

```
"OatN aGO i ReIe stINSSiGJ iG the sDUIeZe UNGtiCC's JaIOeGs at the KatiHaGaGO Re ReGt CINZ
tNUiH tN tNUiH ZaIKeSiGJ at the iGJeGDitF that ZeGshNReO iG KaIiNDs eGteIUIises, tiSS OatN
JaKe eYUIessiNG tN his RaIZaOZiIatiNG CNI thNse ZeG RhN HaG eYUSNit Rhat aIe HaSSeO 'HiUheIs.'
"sN RINte SeNG Aattista aSAeIti GeaI the AeJiGGiGJ NC the sDHHiGHt ADtsDJJestiKe RNIL
that eaIGeO hiZ the titSe NC CatheI NC ResteIGHIFUtNSNJF. aSAeIti Ras the CiIst NC a JINDU
NC RIiteIs RhN, eSeZeGt AFeSeZeGt, OeKeSNUeO a tFUe NC HiUheI tN RhiHh ZNst NC tNOaF's sFsteZs
```

Step 4: G → n, O → d

The word `TGO` (5 times) now reads `aGO`. The common English word that fits is “and”, so G is **n** and O is **d**. It is confirmed right away by `XG`, the third most frequent two-letter word (11 times), which becomes `in`.

Command in the tool: `G=n O=d`. First lines of the text after this step:

```
"datN and i ReIe stINSSinJ in the sDUIeZe UNntiCC's JaIdens at the KatiHanand Re Rent CINZ
tNUiH tN tNUiH ZaIKeSinJ at the inJenDitF that ZenshNRed in KaIiNDs enteIUIises, tiSS datN
JaKe eYUIessiNn tN his RaIZadZiIatiNn CNI thNse Zen RhN Han eYUSNit Rhat aIe HaSSed 'HiUheIs.'
"sN RINte SeNn Aattista aSAeIti neaI the AeJinninJ NC the sDHHinHt ADtsDJJestiKe RNIL
that eaIned hiZ the titSe NC CatheI NC ReSteInHIFUtNSNJF. aSAeIti Ras the CiIst NC a JINDU
NC RIiteIs RhN, eSeZent AFeSeZent, deKeSNUed a tFUe NC HiUheI tN RhiHh ZNst NC tNdaF's sFsteZs
```

Step 5: N → o, C → f

The most frequent two-letter words are `NC` (24 times) and `WN` (13 times). `WN` now reads `tN`, which can only be “to”, so N is **o** (N is also 4th by frequency, exactly like O in English). Then `NC` reads “oC”, and the most common two-letter English word starting with o is “of”, so C is **f**.

Command in the tool: `N=o C=f`. First lines of the text after this step:

```
"dato and i ReIe stIoSSinJ in the sDUIeZe Uontiff's JaIdens at the KatiHanand Re Rent fIoZ
toUiH to toUiH ZaIKeSinJ at the inJenDitF that ZenshoRed in KaIioDs enteIUIises, tiSS dato
JaKe eYUIession to his RaIZadZiIation foI those Zen Rho Han eYUSoit Rhat aIe HaSSed 'HiUheIs.'
"so RIote Seon Aattista aSAeIti neaI the AeJinninJ of the sDHHinHt ADtsDJJestiKe RoIL
that eaIned hiZ the titSe of fatheI of ReSteInHIFUtoSoJF. aSAeIti Ras the fiIst of a JIoDU
of RIiteIs Rho, eSeZent AFeSeZent, deKeSoUed a tFUe of HiUheI to RhiHh Zost of todaF's sFsteZs
```

Step 6: R → w

The word `RQN` (8 times) now reads `Rho`, which is “who”, so R is **w**. This is confirmed by `RTP` (also 8 times), which becomes `was`.

Command in the tool: `R=w`. First lines of the text after this step:

```
"dato and i weIe stIoSSinJ in the sDUIeZe Uontiff's JaIdens at the KatiHanand we went fIoZ
toUiH to toUiH ZaIKeSinJ at the inJenDitF that Zenshowed in KaIioDs enteIUIises, tiSS dato
JaKe eYUIession to his waIZadZiIation foI those Zen who Han eYUSoit what aIe HaSSed 'HiUheIs.'
"so wIote Seon Aattista aSAeIti neaI the AeJinninJ of the sDHHinHt ADtsDJJestiKe woIL
that eaIned hiZ the titSe of fatheI of westeInHIFUtoSoJF. aSAeIti was the fiIst of a JIoDU
of wIiteIs who, eSeZent AFeSeZent, deKeSoUed a tFUe of HiUheI to whiHh Zost of todaF's sFsteZs
```

Step 7: H → c, U → p, I → r

At the end of the first sentence there is the phrase `what aIe HaSSed 'HiUheIs.'`. The words "what aIe HaSSed …" look like "what are called", and since the text is about cryptography, the word in quotes `HiUheIs` should be "ciphers". This gives H = **c**, U = **p** and I = **r** (I is 7th by frequency, R is 9th in English).

Command in the tool: `H=c U=p I=r`. First lines of the text after this step:

```
"dato and i were stroSSinJ in the sDpreZe pontiff's Jardens at the Katicanand we went froZ
topic to topic ZarKeSinJ at the inJenDitF that Zenshowed in KarioDs enterprises, tiSS dato
JaKe eYpression to his warZadZiration for those Zen who can eYpSoit what are caSSed 'ciphers.'
"so wrote Seon Aattista aSAerti near the AeJinninJ of the sDccinct ADtsDJJestiKe worL
that earned hiZ the titSe of father of westerncrFptoSoJF. aSAerti was the first of a JroDp
of writers who, eSeZent AFeSeZent, deKeSoped a tFpe of cipher to which Zost of todaF's sFsteZs
```

Step 8: F → y, S → l, J → g, D → u

The word `HIFUWNSNJF` appears 3 times and now reads `crFptoSoJF`, which is clearly "cryptology": F = **y**, S = **l**, J = **g**. The doubled `SS` (11 times) becomes "ll", for example `PWINSSXGJ` → `strolling`. Then `FND` (6 times) reads "yoD", which is "you", so D is **u**.

Command in the tool: `F=y S=l J=g D=u`. First lines of the text after this step:

```
"dato and i were strolling in the supreZe pontiff's gardens at the Katicanand we went froZ
topic to topic ZarKeling at the ingenuity that Zenshowed in Karious enterprises, till dato
gaKe eYpression to his warZadZiration for those Zen who can eYploit what are called 'ciphers.'
"so wrote leon Aattista alAerti near the Aeginning of the succinct AutsuggestiKe worL
that earned hiZ the title of father of westerncryptology. alAerti was the first of a group
of writers who, eleZent AyeleZent, deKeloped a type of cipher to which Zost of today's systeZs
```

Step 9: Z → m, A → b, K → v, Y → x, L → k, E → j, B → q, M → z

Only the rare letters Z, A, K, L, Y, E, M and B are left. Most words are readable now, so I guessed them from words that miss only one or two letters:

- `ZVG` → `Zen` → “men”, so Z = **m**;
- `TSAVIWX` → `alAerti` → “alberti” (the name appears 7 times), so A = **b**;
- `KTWXHTG` in “the `Katican` gardens” → “Vatican”, so K = **v**;
- `VYUIVPPXNG` → `eYpression` → “expression”, so Y = **x**;
- `LGNR` → `Lnow` → “know”, so L = **k**;
- `ENQG` → `Eohn` → “John” (John Symonds), so E = **j**;
- `CIVBDVGHF` → `freBuency` → “frequency”, so B = **q**;
- `VUXWNZXMVP` → `epitoZiMes` → “epitomizes”, so M = **z**.

Command in the tool: `Z=m A=b K=v Y=x L=k E=j B=q M=z`. First lines of the text after this step:

```
"dato and i were strolling in the supreme pontiff's gardens at the vaticanand we went from
topic to topic marveling at the ingenuity that menshowed in various enterprises, till dato
gave expression to his warmadmiration for those men who can exploit what are called 'ciphers.'
"so wrote leon battista alberti near the beginning of the succinct butsuggestive work
that earned him the title of father of westerncryptology. alberti was the first of a group
of writers who, element byelement, developed a type of cipher to which most of today's systems
```

4.3. The decrypted message

After step 9 all 26 letters are known, and the `plain` command prints the whole message (Figure 5).



Figure 5. *The complete key and the decrypted message in the tool*

The decrypted text of variant 7 is:

```
"DATO and I were strolling in the Supreme Pontiff's gardens at the Vaticanand we went from
topic to topic marveling at the ingenuity that menshowed in various enterprises, till Dato
gave expression to his warmadmiration for those men who can exploit what are called 'ciphers.'
"So wrote Leon Battista Alberti near the beginning of the succinct butsuggestive work
that earned him the title of Father of WesternCryptology. Alberti was the first of a group
of writers who, element byelement, developed a type of cipher to which most of today's systems
ofcryptography belong. The species is polyalphabetic substitution.It was the amateurs of
cryptology who created the species. Theprofessionals, who almost certainly surpassed them in
cryptanalyticexpertise, concentrated on the down-to-earth problems of the systemsthat
were then in use but are now outdated. The amateurs, unfettered tothese realities, soared
into the empyrean of theory. There were fourwhose thought took wings: a famous architect, an
intellectual cleric, anecclesiastical courtier, and a natural scientist.The architect was
Alberti, a man who, perhaps better than anyoneexcept Leonardo da Vinci, epitomizes the
Renaissance ideal of theuniversal man. Born in 1404, the illegitimate but favored son of a
familyof rich Florentine merchants, Alberti enjoyed extraordinary intellectualand athletic
aptitudes. He painted, composed music, and was regardedas one of the best organists of his day.
Writings poured from his pen. HisDe Re Aedificatoria, the first printed book on
architecture, written whileGothic churches were still being built, helped shape the
thoughts ofthose who built such utterly non-Gothic structures as St. Peter's Basilicain
Rome. Jacob Burckhardt, author of the classic The Civilization of theRenaissaance in Italy,
singled out Alberti as one of the truly all-sidedmen who tower above their numerous many-sided
contemporaries. Andanother great Renaissance scholar, John Symonds, declared that
"Hepresents the spirit of the 15th century at its very best."Among his friends was the
pontifical secretary, Leonardo Dato, one ofthe learned men of his age, who during that memorable
stroll in theVatican gardens brought the conversation around to cryptology. "You'vealways been
interested in these secrets of nature," Dato said. "What doyou think of these decipherers?
Have you tried your hand at it, as muchas you know how to?"Alberti smiled. He knew that Date's
duties included ciphers (it wasbefore the curia had a separate cipher secretary). "You're the
head of thepapel secretariat," he teased. "Could it be that you had to use thesethings a few
times in matters of great importance to His Holiness?""That's why I brought it up," Dato
replied candidly. "And because ofthe post I have, I want to be able to do it myself without
having to useoutside interpreters. For when they bring me letters in cipher interceptedby
spies, it's no joking matter. So please—if you've thought up any newideas having to do with
this business, tell me about them." So Albertipromised that he would do some work on it so
that Dato would see thatit was profitable to have asked him, and the result was the essay
that hewrote in 1466 or early 1467, when he was 62 or 63.He implied that he thought up the
idea of frequency analysis all byhimself, but the conception that he set forth is far too
matured for that.Nevertheless, his remarkably lucid Latin essay, totaling about 25manuscript
pages, constitutes the West's oldest extant text oncryptanalysis.
```

The text is about Leon Battista Alberti, the “Father of Western Cryptology”, and his essay on ciphers, which is the oldest known Western text on cryptanalysis (it looks like a fragment from David Kahn’s book *The Codebreakers*). Some words are glued together (“Vaticanand”, “men-showed”) and there are a few typos (“Renaissaance”, “Date’s”). These are already in the ciphertext, the decryption only keeps them.

4.4. The recovered key

Putting the plaintext alphabet next to the cipher letters that replace each plaintext letter gives the encryption key (Table 3). Table 4 shows the same key in the other direction, which is what is needed for decryption.

Plain	**a**	**b**	**c**	**d**	**e**	**f**	**g**	**h**	**i**	**j**	**k**	**l**	**m**	**n**	**o**	**p**	**q**	**r**	**s**	**t**	**u**	**v**	**w**	**x**	**y**	**z**
Cipher	**T**	**A**	**H**	**O**	**V**	**C**	**J**	**Q**	**X**	**E**	**L**	**S**	**Z**	**G**	**N**	**U**	**B**	**I**	**P**	**W**	**D**	**K**	**R**	**Y**	**F**	**M**
x	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

Table 3. *Reconstructed encryption alphabet (plaintext letter → ciphertext letter)*

Cipher	**A**	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**	**J**	**K**	**L**	**M**	**N**	**O**	**P**	**Q**	**R**	**S**	**T**	**U**	**V**	**W**	**X**	**Y**	**Z**
Plain	**b**	**q**	**f**	**u**	**j**	**y**	**n**	**c**	**r**	**g**	**v**	**k**	**z**	**o**	**d**	**s**	**h**	**w**	**l**	**a**	**p**	**e**	**t**	**i**	**x**	**m**

Table 4. *Decryption alphabet (ciphertext letter → plaintext letter)*

Looking at Table 3, I noticed that the cipher letters are not random: from one plaintext letter to the next the cipher letter always moves 7 positions forward in the alphabet (a→T = 19, b→A = 0, c→H = 7, d→O = 14, …). So the key is actually an **affine cipher**:

$$c = (7 \cdot x + 19) \bmod 26,$$

where x is the position of the plaintext letter (a = 0, …, z = 25). I checked this formula for all 26 letters in the unit tests. This means that once the type of the cipher is known, only two letter pairs are enough to find the key. For example, from the first step, e (4) → V (21) and t (19) → W (22) give $4a + b \equiv 21$ and $19a + b \equiv 22 \pmod{26}$. Subtracting gives $15a \equiv 1$, so $a = 7$ (because $15 \cdot 7 = 105 = 4 \cdot 26 + 1$) and $b = 21 - 28 \equiv 19$.

4.5. How far frequencies alone get

To see how important the word patterns were, I compared the result with the simplest possible attack: matching the two sorted orders from Table 2 directly (the most frequent cipher letter with E, the second with T and so on). Only **10 of 26** letters come out correct this way (V→e, W→t, T→a, N→o, X→i, Q→h, H→c, D→u, K→v, L→k). The first five letters are matched perfectly, but already at the 6th place it goes wrong (P stands for s, not for n), and in the middle and the tail of the table most letters are wrong, because their frequencies are too close to each other. So, as the lab says, frequency analysis gives a good start, but the rest has to be done with short words, doubled letters and the context.

5. Conclusions

In this laboratory work I broke a monoalphabetic substitution cipher using frequency analysis, without knowing anything about the key. The intercepted text had about 2,800 letters, which was more than enough: the frequencies of the cipher letters were almost the same as the frequencies of English letters, and the five most frequent cipher letters (V, W, T, N, X) corresponded exactly to E, T, A, O, I.

The frequencies alone did not solve the cipher. Matching the two sorted tables directly gives only 10 correct letters out of 26. The most useful clues after the frequencies were the short words: “the” (34 times), the one-letter words “a” and “I”, “and”, “of”, “to”, “who”. The possessive ’s after an apostrophe and the doubled letters ss and ll also helped. The last rare letters (b, v, x, k, j, q, z) were found only from the context, by reading words like “Alberti”, “Vatican”, “expression” or “frequency” that were missing just one letter. In total I needed 9 steps.

The main conclusion is that a huge key space does not make a cipher secure. A general monoalphabetic cipher has 26! keys, many more than the Caesar cipher from Lab 1, but it falls just as easily, because every letter is always replaced by the same letter and the statistics of the language stay visible in the ciphertext. The recovered key even turned out to be an affine cipher, which has only $12 \cdot 26 = 312$ keys and can be found from two letter pairs. To resist this kind of attack, a cipher must hide the frequencies, for example by using several alphabets (polyalphabetic ciphers). It is a nice coincidence that this is exactly what Alberti, the subject of my decrypted text, proposed in the 15th century.

I also learned that this attack still needs a person: the computer can count and substitute instantly, but deciding that “crFptoSoJF” is “cryptology” requires knowing the language and the topic of the text.

6. Source code

The full source code (`frequency_analysis.py`, `test_frequency_analysis.py`), the ciphertext and the decrypted text are available in the GitHub repository:

https://github.com/httpcatalin/cs-labs (folder `lab2`)